

API DOCUMENTATION

1. How does each API work?

Matchmaking API

The Matchmaking API is responsible for pairing users as pen pals. It handles creating new matches, retrieving all matches for a user updating match statuses (active, completed, blocked), and deleting matches. Whenever a user signs up or requests a new connection this API determines which users are compatible and stores the match information in the database.

Message API

The Message API manages all user communications. It allows sending messages retrieving messages in a match and deleting messages. This ensures that users can exchange text messages in real-time or near real-time. Each message is linked to a match and contains information about the sender, recipient, content, and timestamp.

Profile API

The Profile API allows users to create, update, retrieve, or delete their cultural profiles. These profiles include information such as hobbies, languages, and other cultural notes that help improve matchmaking suggestions. This API ensures that users' cultural data is stored securely and can be accessed when generating matches.

Moderation API

The Moderation API is used for reporting inappropriate behaviour and reviewing reported users. Users can file reports against other users, and administrators can retrieve, review, and take action based on these reports. This API helps maintain a safe and respectful environment within the app.

2. How to test each API using curl?

Matchmaking API

Create a match:

```
curl -X POST https://our-project-id/api/matches \
-H "Content-Type: application/json" \
```

```
-H "Authorization: Bearer <idToken>" \  
  
-d '{  
  
  "user1Id": "abc123",  
  
  "user2Id": "def456",  
  
  "status": "active"  
  
}'
```

Get all matches for a user:

```
curl https://our-project-id/api/users/abc123/matches
```

Message API

Send a message:

```
curl -X POST https://our-project-id/api/matches/match789/messages \  
  
-H "Content-Type: application/json" \  
  
-H "Authorization: Bearer <idToken>" \  
  
-d '{  
  
  "senderId": "abc123",  
  
  "recipientId": "def456",  
  
  "text": "Hello!"  
  
}'
```

Get messages in a match:

```
curl https://our-project-id/api/matches/match789/messages
```

Profile API

Create/Update a cultural profile:

```
curl -X POST https://our-project-id/api/culturalProfiles \  
  
-H "Content-Type: application/json" \  
  

```

```
-H "Authorization: Bearer <idToken>" \  
-d '{  
  "userId": "abc123",  
  "languages": ["English", "Zulu"],  
  "hobbies": ["reading", "cycling"],  
  "culturalNotes": "Enjoys traditional music"  
'
```

Get a cultural profile:

```
curl https://our-project-id/api/culturalProfiles/abc123
```

Moderation API

File a report:

```
curl -X POST https://moderation-api.onrender.com/reports \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <idToken>" \  
-d '{  
  "reportingUserId": "abc123",  
  "reportedUserId": "def456",  
  "reason": "Inappropriate behaviour"  
'
```

Get all reports (admin):

```
curl https://moderation-api.onrender.com/reports \  
-H "Authorization: Bearer <idToken>"
```

3. Where was each API deployed?

- **Matchmaking API:** Publicly hosted (deployed on Render).

- **Message API:** Publicly hosted (deployed on Render).
- **Profile API:** Publicly hosted (deployed on Render).
- **Moderation API:** Publicly hosted on Render (as verified).

4. How did we connect Render and GitHub for this API?

- The project code for each API was stored in a GitHub repository.
- Render was connected to the GitHub repository so that every push to the main branch automatically triggered a deployment.
- This allows the APIs to stay updated in real-time with the latest code from GitHub.

5. How to use each API as our group?

- Each group member can access the API using the base URL and endpoints.
- Members need a Firebase authentication token to perform actions like sending messages, creating matches, or filing reports.
- Team members can test APIs using curl commands or Postman.
- Our group also has access credentials stored securely to manage and monitor the APIs.

6. How can other groups use our API?

- Other groups can access the APIs via the public endpoints provided (Render-hosted URLs).
- They will need a valid Firebase token if authentication is required.
- Example: They can test the Matchmaking API by sending requests to the same endpoints with their own data.
- We can provide API documentation and example curl commands to help other groups integrate or test the APIs easily.

7. Where did our group currently store this API for access?

- All APIs are stored and deployed publicly on Render.
- Source code for the APIs is in a GitHub repository that the group maintains.
- Group members have access to both GitHub (for code) and Render (for deployed endpoints) for testing, debugging, and updating.